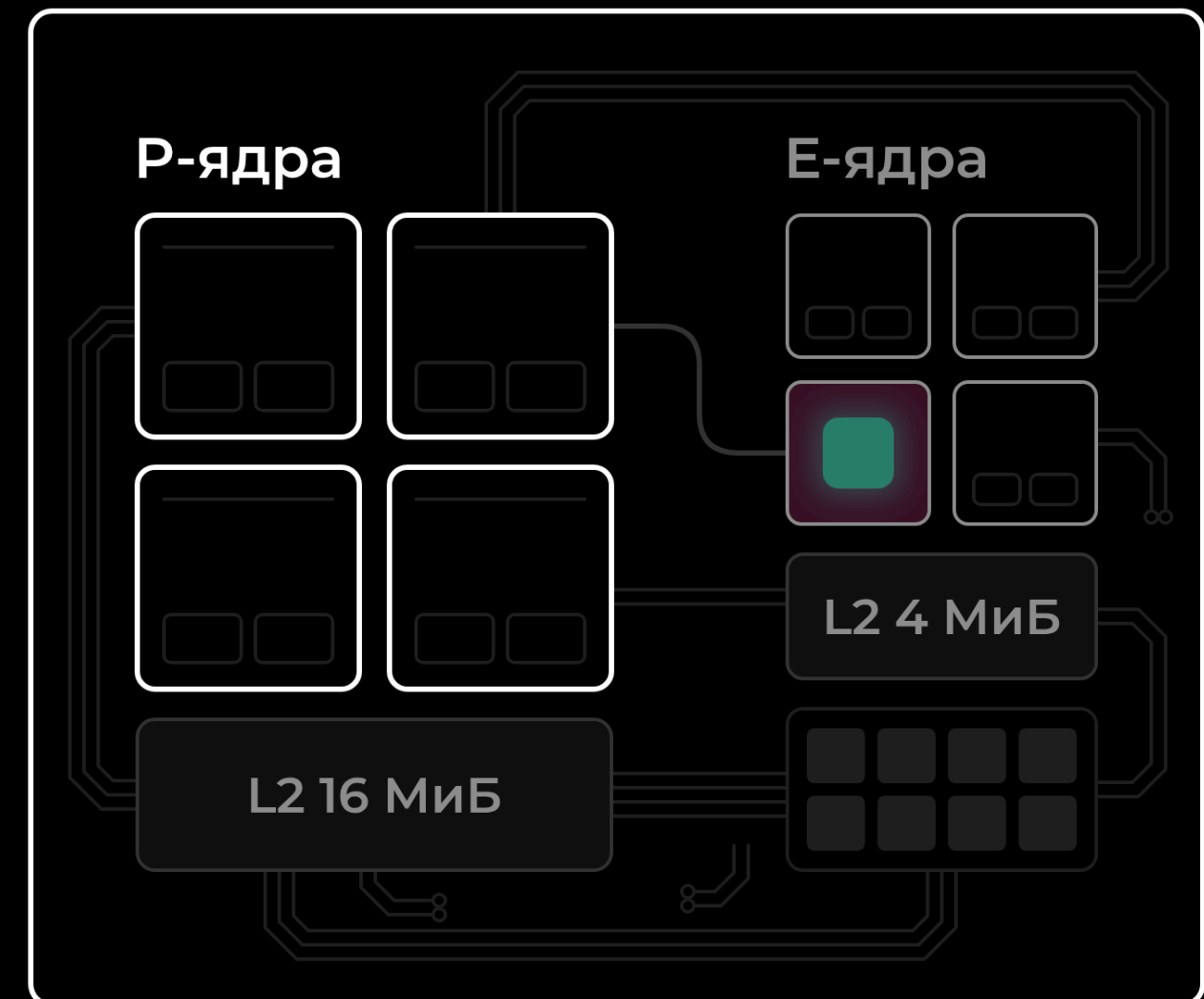


$O(n)$ ПРОИГРАЛ $O(n \log n)$

Ваш алгоритм не выбирал
процессор, на котором работает

Никита Нагорнов

iOS, компания «Исходный код»





НИКИТА НАГОРНОВ

iOS · «Исходный код»

Алгоритмы. Компиляторы. Айфоны.

КОМУ ПРИГОДИТСЯ ЭТОТ ДОКЛАД



Разработчикам

Выбираете алгоритмы
и контейнеры

Тимлидам и ревьюерам

Проверяете перф-изменения
на ревью

Авторам перф-тестов

Мерите в CI
и на целевом устройстве

Что выбрать — и как проверить, что стало быстрее

КОД-РЕВЬЮ, ВТОРНИК, 17:40



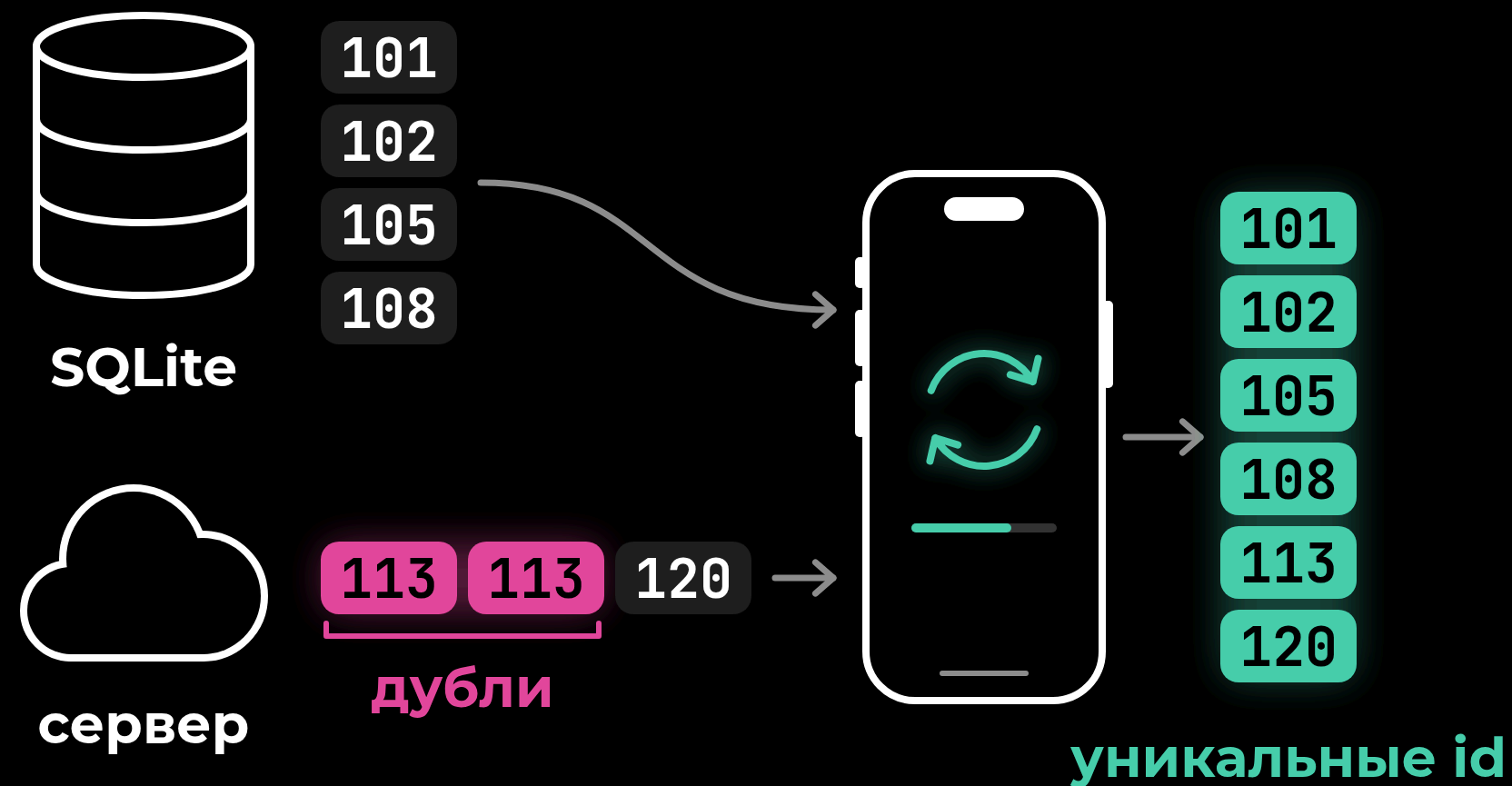
```
// dedup ids after offline sync
- std::sort(ids.begin(), ids.end());
- ids.erase(std::unique(ids.begin(), ids.end()), ids.end());
+ std::unordered_set<uint64_t> s(ids.begin(), ids.end());
+ ids.assign(s.begin(), s.end());
```

Коллега: «**Линейно** вместо $n \log n$ »

Approve ✓

С ЧЕГО ВСЁ НАЧАЛОСЬ

ЗАДАЧА: СКЛЕИТЬ ЛОКАЛЬНЫЕ id И ДЕЛЬТУ С СЕРВЕРА БЕЗ ПОВТОРОВ



1 Локальные id отсортированы
и уникальны

2 Ключ — uint64_t

3 QoS utility

«не срочно»

ВАША СТАВКА

$n \log n$ ПРОТИВ ОЖИДАЕМОГО n



A

sort + unique

$O(n \log n)$

B

unordered_set

$O(n)$ в среднем

C

сначала спросил бы
условие

$\log_2 10^6 \approx 20$

1М 64-битных id · все
разные · один поток ·
порядок не важен

Что первым отправим на замер?

СТАВКА №1 · ЧТО ПОКАЗАЛ СЕКУНДОМЕР

ПЕРВЫЙ ЗАМЕР



нс на элемент · 1М уникальных id · Mac M2 Pro · меньше — лучше

sort + unique



19,9

× 2,6

unordered_set



51,3

1 Память

2 Реализация

3 Данные

4 Процессор

ПЕРЕД ЧЕТЫРЬМЯ РАУНДАМИ

ПРАВИЛО: В ТАЙМЕРЕ ВСЯ ОПЕРАЦИЯ



```
auto v = input;           // вне таймера  
timer.start(); alg(v); timer.stop();  
if (sorted(v) != ref) abort(); // и в релизе
```

в таймере:

reserve

вставки

выгрузка

деструктор

**Быстрое, но неправильное решение
в турнире не участвует**

ХЕШ ДОРОЖАЕТ ПОСЛЕ 512К



нс на элемент · ниже — лучше



УЗЛЫ СТОЯТ АЛЛОКАЦИЙ И ПРЫЖКОВ



sort + unique

`vector<uint64_t>`



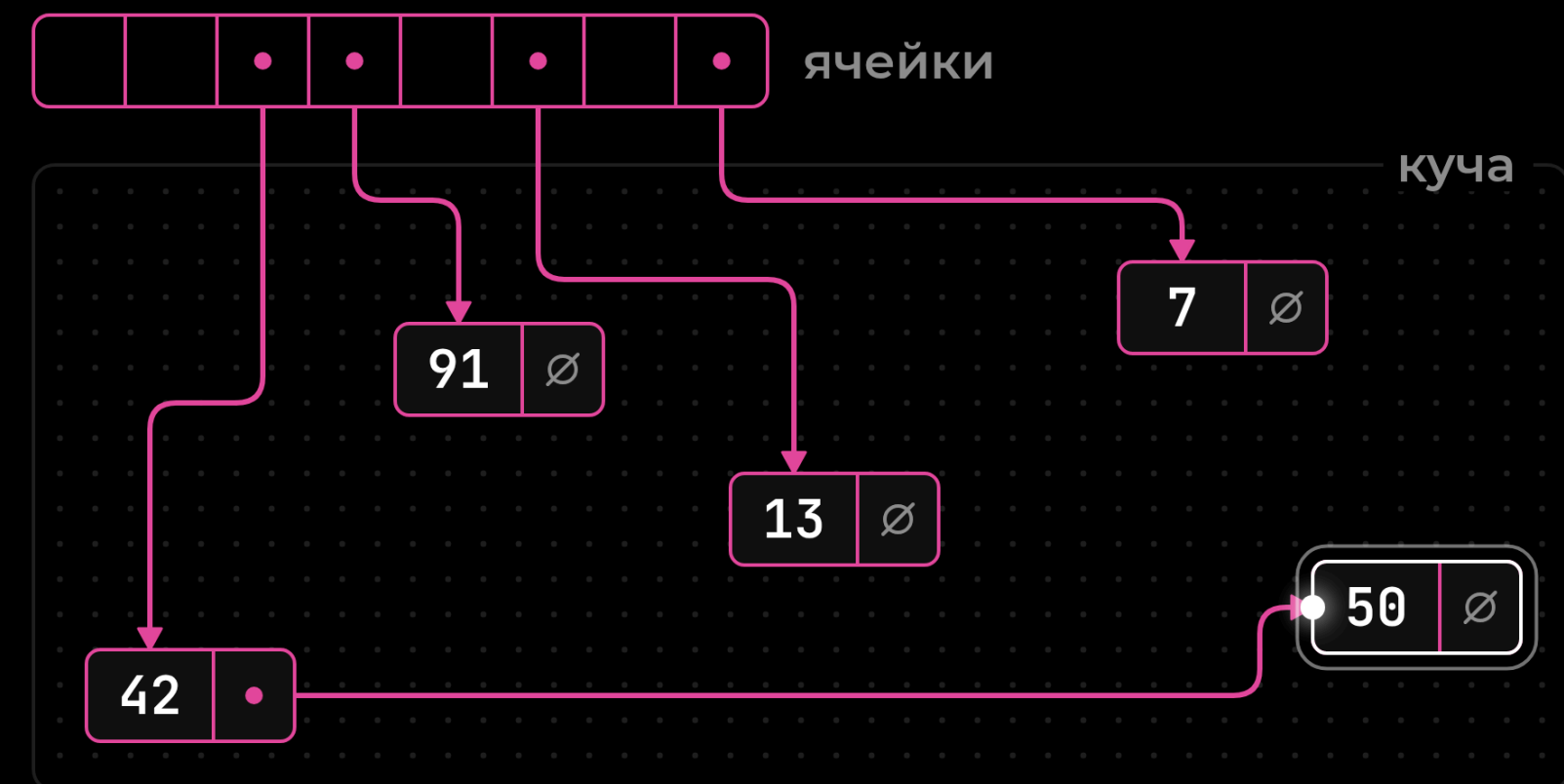
8 Б на ключ, подряд

Узлы: адрес следующего
узнаём после чтения текущего

0 new

Вход готов. $N = U = 2^{20}$, `libc++`

unordered_set



≈1 млн new

≈42 МБ с округлением `malloc`

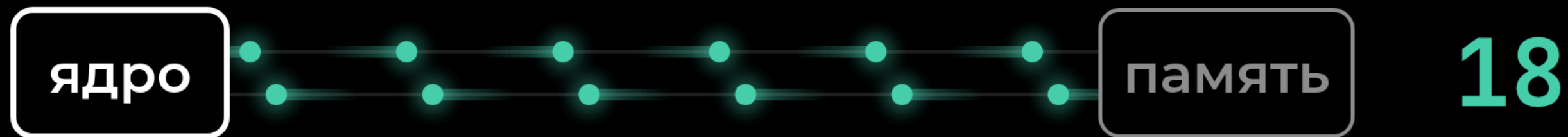
ПРОЦЕССОР МОЖЕТ ЖДАТЬ КАЖДЫЙ ШАГ



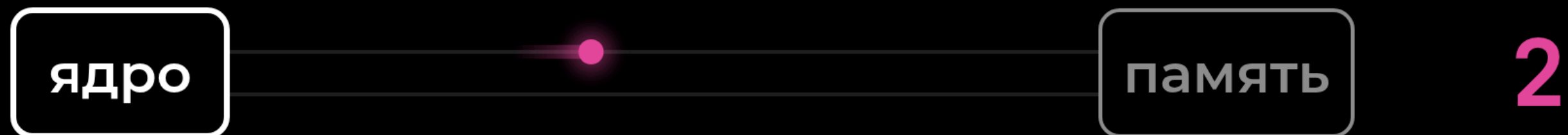
Время $\approx N \times$ шагов на элемент $\times$ цена шага

Упрощённая модель: число шагов растёт, цена шага зависит от машины

адреса известны заранее



адрес внутри узла

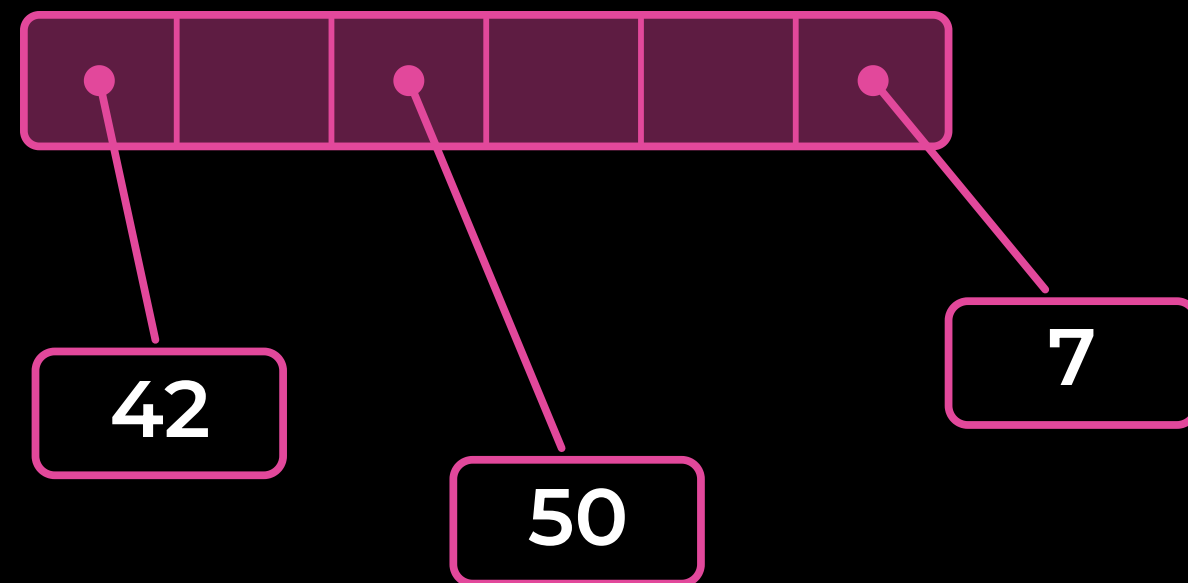


Независимые чтения перекрываются. Зависимые ждут друг друга

ПЛОСКАЯ ТАБЛИЦА: КЛЮЧИ В МАССИВЕ



unordered_set

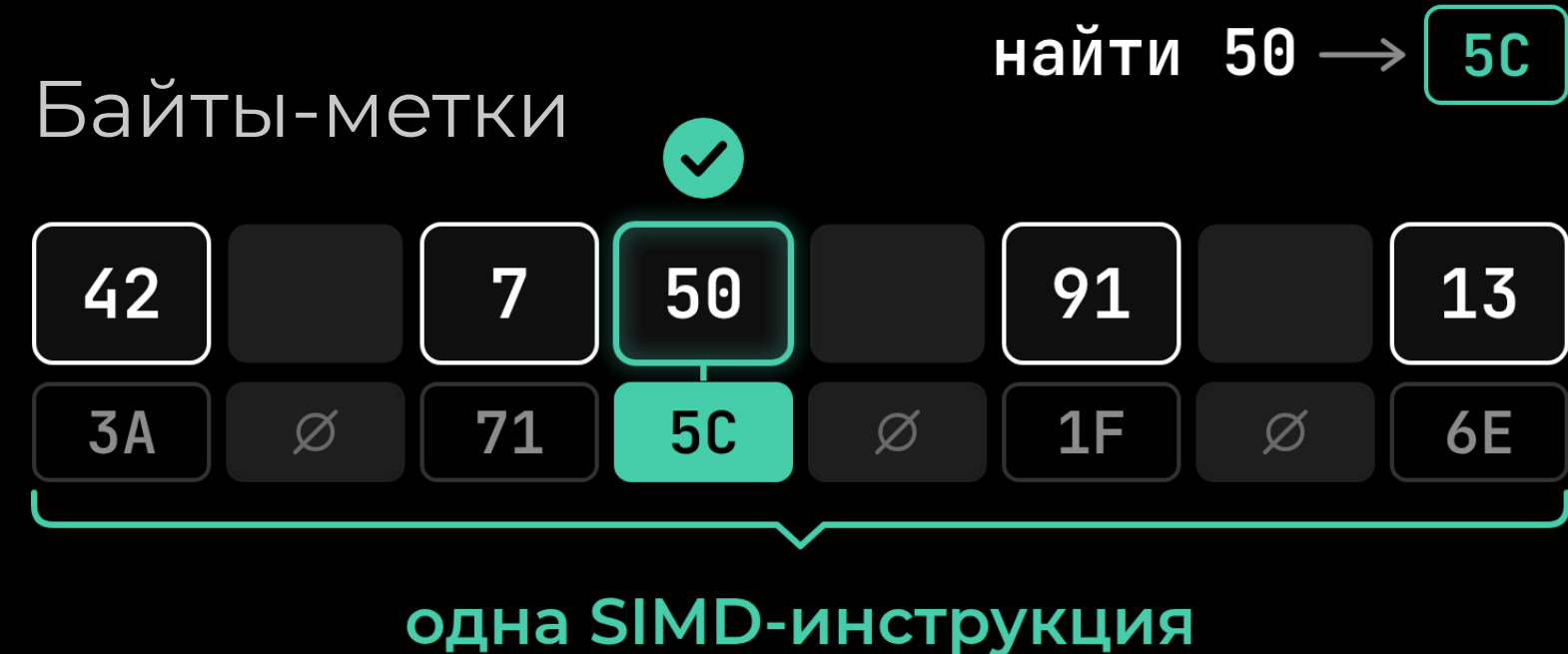


Свой new на каждый ключ

Тот же принцип:

map в Go ≥ 1.24 , HashMap в Rust

flat_hash_set · Swiss Table



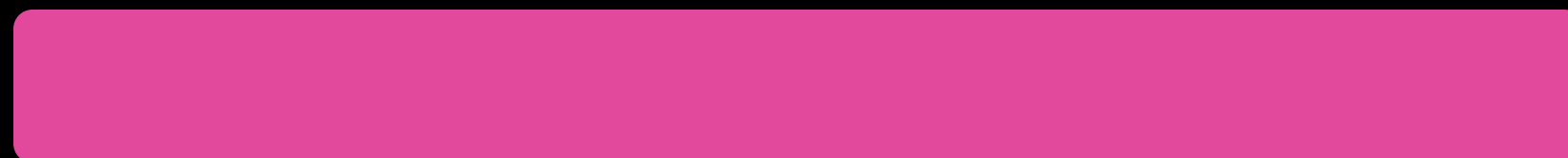
- + Одна аллокация на всю таблицу
- Ссылки не переживают рост

ТРИ РЕАЛИЗАЦИИ, ОДИН ВХОД



нс на элемент · 1М уникальных id · M2 Pro · меньше — лучше

unordered_set



51,3

sort + unique



19,9

flat_hash_set



11,7

×1,7

flat против sort + unique

radix + unique: 6,9 нс/эл

Только целые ключи. Дополнительный буфер на N

СОРТИРОВКА ИЛИ ХЕШ?



точка отсчёта — ставка №1: миллион id, все разные

sort + unique



20

unordered_set



51

×2,6

Тот же миллион.
Уникальных — 1%.
Кто впереди?

sort

хеш

ПОБЕДИТЕЛЯ НАЗНАЧАЮТ ДАННЫЕ



нс на элемент · $N = 2^{20}$ · M2 Pro · меньше — лучше

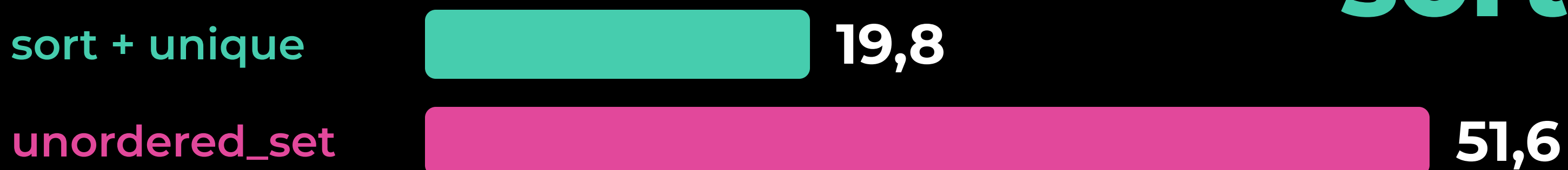
Одинаковая линейная шкала полос: 0–60 нс/эл

1% уникальных



xesh ×3,7

100% уникальных



sort ×2,6

ТРЕБОВАНИЕ В ТИКЕТЕ МЕНЯЕТ ПОБЕДИТЕЛЯ



что нужно на выходе · нс на элемент

Уникальные, порядок любой

sort 20,2 · хеш 22,6

≈ близко

Отсортированные

sort 20,2 · хеш 32,6

sort ×1,6

В порядке первого появления

sort 74,6 · хеш 22,0

хеш ×3,4

«Порядок любой» = любой **каждый раз**:

flat_hash_set, Set в Swift и map в Go меняют его от запуска к запуску

ПОРЯДОК ТОГО ЖЕ ВХОДА



те же ключи и повторы, другой порядок входа · нс на элемент

Случайный

sort 20,2 · хеш 22,6

≈ близко

Сериями одинаковых

sort 20,5 · хеш 15,5

хеш ×1,3

Уже отсортированный

1

sort 2,0 · хеш 16,5

sort ×8

libc++: ≈2 сравнения на элемент вместо ≈22

libstdc++ так не делает

ВОСПРОИЗВОДИМЫЙ РЕЗУЛЬТАТ ТОЖЕ МОЖЕТ ОБМАНУТЬ



sort + unique · 1024 id · нативная macOS, M2 Pro · нс/элемент

6,8

Один и тот же вход

14,4

64 разных входа

Контроль на x86: одинаковые входы быстры
даже по разным адресам

Повторяемость и проверка результата ещё не проверяют
модель опыта

МЕНЯЕМ QoS ПОТОКА — МЕНЯЕТСЯ РЕЗУЛЬТАТ



```
pthread_set_qos_class_self_np(  
    QOS_CLASS_UTILITY, 0);
```

512K id · половина уникальных · sort / unordered_set, нс на элемент

userInitiated

utility

Mac
M2 Pro

≈ близко
19,6 / 18,7

≈ близко
20,1 / 21,0

iPhone 14 Pro
A16 Bionic

≈ близко
19,6 / 18,2

sort ×1,9
55,3 / 105,3

Mac, 1M: преимущество sort тоже выросло — ×1,3 → ×1,8

РАУНД 4 ИЗ 4 · ПРОЦЕССОР

НА МИЛЛИОНЕ id СОРТИРОВКА ВПЕРЕДИ В
3,3 РАЗА

НА
RD fest

iPhone 14 Pro · A16 Bionic · QoS utility

×3,3

sort + unique
быстрее unordered_set

нс / элемент · меньше — лучше

sort + unique

57,4

unordered_set

187,7

1M id · половина уникальных · одинаковые входы

КРИВУЮ МЕНЯЮТ ЧАСТОТА И РАЗМЕР КЭША



Р

Производительное
ядро

Свой кэш и своя частота

Е

Энергоэффективное
ядро

Свой кэш и своя частота

У разных типов ядер — разные параметры памяти

Частота

меняет высоту кривой

Размер кэша

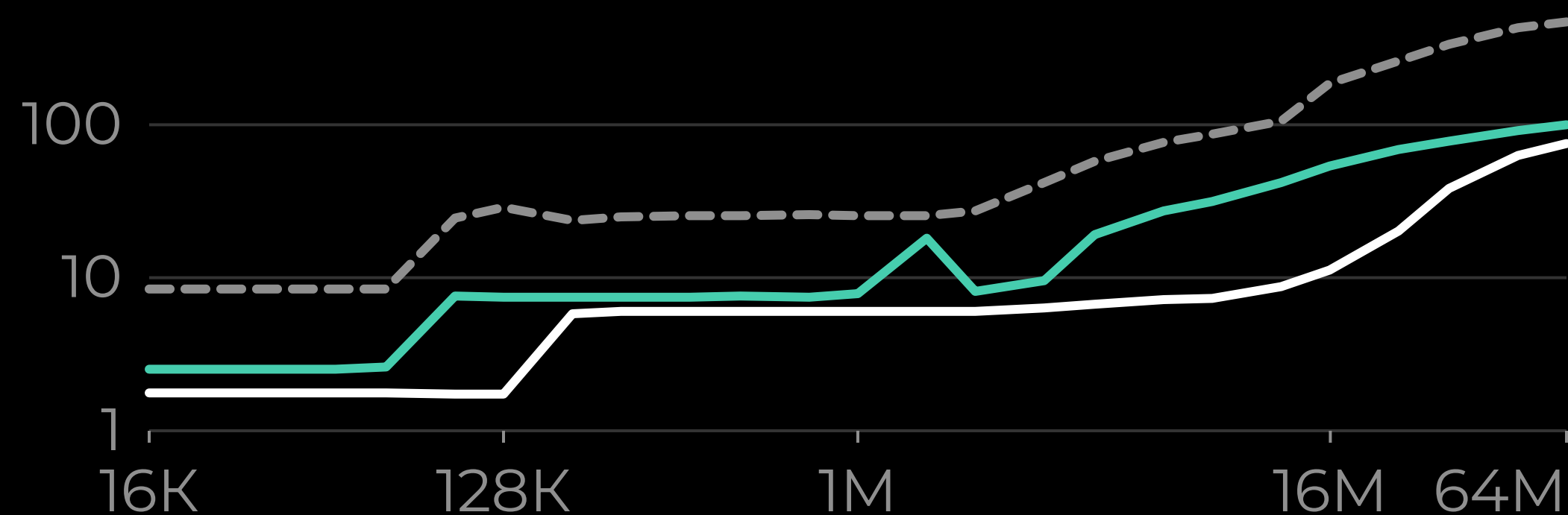
меняет положение излома

ИЗЛОМЫ ПОХОЖИ НА СМЕНУ ТИПА ЯДРА



iPhone 14 Pro · A16 Bionic

нс на чтение · КиБ/МиБ · лог. шкалы



userInitiated

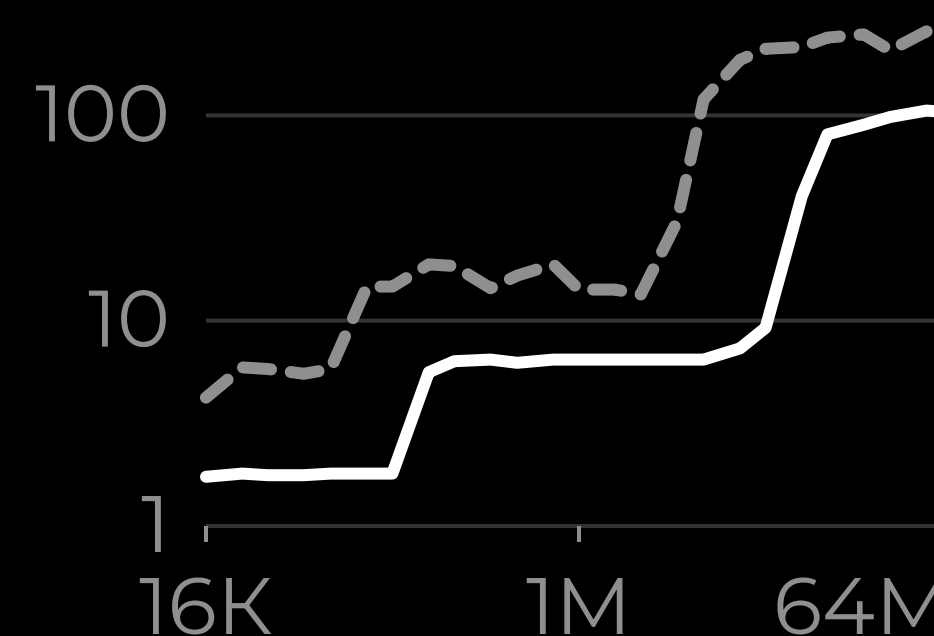
background

utility

utility-кривая похожа на E-ядро; это косвенный признак

Mac · M2 Pro

тот же метод



ОДИН БЕНЧМАРК — РАЗНЫЕ ОТВЕТЫ



sort + unique против unordered_set · $U = N/2 \cdot x$ — выигрыш по времени

N	Linux x86 повтор · Xeon VM	Mac M2 Pro userInitiated	iPhone 14 Pro QoS utility
64K	хеш ×1,64	≈ близко	≈ близко
256K	хеш ×1,29	≈ близко	sort ×1,3
512K	хеш ×1,12	≈ близко	sort ×1,9
1M	≈ близко	sort ×1,3	sort ×3,3

CI отвечает про свою среду. Пользовательский код живёт в другой.

СОРТИРОВКА ДЕЛЬТЫ И СЛИЯНИЕ



sort
дельты 64K

unique

слить
с локальными 1M

Уникальные id

```
std::sort(D.begin(), D.end());
```

```
D.erase(std::unique(D.begin(), D.end()), D.end());
```

```
std::set_union(L.begin(), L.end(), D.begin(), D.end(), out);
```

без unique дубли дельты
останутся

×3,5–6,6

быстрее полной сортировки
Замер на x86

ПРАКТИЧЕСКИЙ ВЫБОР

ЦЕНА В МИЛЛИСЕКУНДАХ



Лабораторная серия · iPhone 14 Pro · QoS utility

512K id · половина уникальных

sort ≈ 29 мс · unordered_set ≈ 55 мс

+26 мс · +12 МиБ

На тестовом 1М: ≈ 60 против ≈ 197 мс

Для нашего примера: сортировать дельту, убрать повторы,
слить

ПОСЛЕ ЧЕТЫРЁХ РАУНДОВ · ПЕРЕНОС НА СВОЮ ЗАДАЧУ

ЭТОТ МЕТОД РАБОТАЕТ ЗА ПРЕДЕЛАМИ iPhone



Бэкенд

Целевой CPU и библиотека как в проде

Мобильное

Реальный executor и QoS на устройстве

Язык

Конкретная реализация контейнера

Название Set не обещает одинакового устройства или цены

КОД-РЕВЬЮ, ВТОРНИК, 17:40



```
// dedup ids after offline sync
- std::sort(ids.begin(), ids.end());
- ids.erase(std::unique(...), ids.end());
+ std::unordered_set<uint64_t> s(...);
+ ids.assign(s.begin(), s.end());
```

Какой
комментарий
вы оставите?

Changes requested

1 Контракт

Какой выход нужен?

2 Данные

Размер, повторы,
порядок входа?

3 Исполнение

Устройство,
библиотека, QoS?

ВАШ АЛГОРИТМ НЕ ВЫБИРАЛ ПРОЦЕССОР



Проверьте, какой ему достался

Начните с одного перф-теста
на целевом устройстве



github.com/Neoron95/conference-hardfest-algorithms